

Углубленный Python

Лекция 9

Профилирование, отладка, логирование

Кандауров Геннадий



education

Напоминание отметиться на портале

+ ОСТАВИТЬ ОТЗЫВ

vk образование

БлогиЛюдиПрограммаВакансииРасписание

Q<

VK

Техно

Открыт приём заявок!

чт, 8 сентября

Нет занятий

пт, 9 сентября

18:00 Углубленный Py... с3
Введение в Python, основные
понятия, тестирование
Г. Кандауров

сб, 10 сентября

Нет занятий

вс, 11 сентября

Нет занятий

пн, 12 сентября

Нет занятий

Углубленный Python

↓ 0 ↑

Блог для материалов по курсу "Углубленный Python"

57 читателей, 2 топика

ПодписатьсяСоздать топик

Поиск по авторам, заголовку и тексту топика...

Найти

Добро пожаловать на курс!

Углубленный Python

ИзменитьУдалить

Всем привет и добро пожаловать на курс по углубленному изучению Python!

Прямой эфир

МоиВсе

Геннадий Кандауров час назад
Углубленный Python → Добро пожаловать
на курс! 0

Екатерина Черкасова 7 дней назад
Стажировка → Приглашаем мобильных,
фронтенд- и бэкэнд-разработчиков на
Weekend Offer! 0

Дарья Вовченко 9 дней назад
Углубленный Python → Добро пожаловать
в образовательные проекты VK
Образование! 0

Дарья Вовченко 9 дней назад
Разработка веб-сервисов на

Квиз про прошлой лекции



Содержание занятия

1. Профилирование
2. Логирование
3. Отладка

Профилирование

Сбор характеристик работы программы, таких как время выполнения отдельных фрагментов (обычно подпрограмм), число верно предсказанных условных переходов, число кэш-промахов и т. д.



Профилирование

Цель:

- найти узкие места в коде

Основные способы:

- CPU
- Память
- Частота/продолжительность вызовов функций

Методы:

- Статистический (сэмплирование)
- Детерминированный (инструментирование)

Профилирование

- cProfile - написанная на C, быстрая реализация профилировщика
- profile - нативная реализация профилировщика на чистом python, значительно медленнее

```
python -m cProfile -o output.txt ptest.py
```

```
import pstats
```

```
p = pstats.Stats("output.txt")
```

```
p.strip_dirs().sort_stats(-1).print_stats()
```

Профилирование

```
import cProfile , pstats , io
```

```
pr = cProfile.Profile()
```

```
pr.enable()
```

```
# ... do something ...
```

```
pr.disable()
```

```
s = io.StringIO()
```

```
sortby = "cumulative"
```

```
ps = pstats.Stats(pr, stream=s).sort_stats(sortby)
```

```
ps.print_stats()
```

```
print(s.getvalue())
```


cProfile

1567629 function calls (1166637 primitive calls) in 809.730 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.164	0.164	809.738	809.738	/Users/project/src/.env3/lib/python3.7/site-packages/tornado/ioloop.py:568(start)
4961	806.444	0.163	806.444	0.163	/Users/project/src/.env3/lib/python3.7/site-packages/tornado/platform/kqueue.py:66(poll)
9982/8005	0.086	0.000	3.095	0.000	/Users/project/src/.env3/lib/python3.7/site-packages/tornado/stack_context.py:269(wrapped)
5657	0.011	0.000	2.767	0.000	/Users/project/src/.env3/lib/python3.7/site-packages/tornado/ioloop.py:471(_run_callback)
6766/2479	0.083	0.000	1.869	0.001	/Users/project/src/.env3/lib/python3.7/site-packages/tornado/gen.py:507(run)
2445	0.009	0.000	1.775	0.001	/Users/project/src/.env3/lib/python3.7/site-packages/tornado/gen.py:567(inner)
2445	0.005	0.000	1.764	0.001	/Users/project/src/.env3/lib/python3.7/site-packages/tornado/gen.py:497(set_result)
430	0.008	0.000	0.902	0.002	/Users/project/src/gekko/net/resolver.py:414(resolve)
75	0.000	0.000	0.669	0.009	/Users/project/src/gekko/handlers2/executor.py:93(callback)
75	0.000	0.000	0.669	0.009	/Users/project/src/gekko/handlers2/executor.py:72(_handler_callback)
48	0.000	0.000	0.669	0.014	/Users/project/src/gekko/handlers2/executor.py:114(_done)
72	0.000	0.000	0.612	0.009	/Users/project/src/gekko/location2.py:266(_call_location_method)
60	0.000	0.000	0.610	0.010	/Users/project/src/gekko/location2.py:91(create_gen_tasks)
63	0.000	0.000	0.609	0.010	/Users/project/src/gekkoapps/gosearch/locations/ajax_web.py:27(get)
9	0.000	0.000	0.576	0.064	/Users/project/src/gekkoapps/common/locations/base.py:104(create_response)
9	0.001	0.000	0.572	0.064	/Users/project/src/gekkoapps/common/locations/base.py:97(render_view)
9	0.000	0.000	0.242	0.027	/Users/project/src/gekkoapps/common/locations/base.py:173(get_data_from_view)
9	0.000	0.000	0.242	0.027	/Users/project/src/gekkoapps/common/views/base.py:136(get_data)
9	0.000	0.000	0.239	0.027	/Users/project/src/gekkoapps/gosearch/v1/web/view/compat.py:14(create_location_data)
9	0.000	0.000	0.238	0.026	/Users/project/src/gekkoapps/gosearch/v1/web/view/produce.py:518(get_data)
9	0.000	0.000	0.220	0.024	/Users/project/src/gekkoapps/common/locations/base.py:183(render_json)
9	0.000	0.000	0.220	0.024	/Users/project/src/gekko/template/helpers.py:148(do_json)
9	0.013	0.001	0.220	0.024	/Users/project/src/.env3/lib/python3.7/site-packages/simplejson/encoder.py:371(encode)
3626	0.030	0.000	0.214	0.000	/Users/project/src/gekko/net/resolver.py:185(resolve)
27	0.000	0.000	0.209	0.008	/Users/project/src/gekkoapps/common/views/serp/v1/creator.py:23(create)

Профилирование памяти

```
pip install memory_profiler
```

```
# run.py
```

```
from memory_profiler import profile
```

```
@profile
```

```
def some_func():
```

```
    lst1 = []
```

```
    lst2 = "1" * 100000
```

```
python -m memory_profiler run.py
```

top/atop

top - консольная команда, которая выводит список работающих в системе процессов и информацию о них.

PID - идентификатор процесса

USER - пользователь, под которым запущен процесс

VRT - объем виртуальной памяти, занимаемой процессом

RES - текущее использование RAM

%CPU - процент доступного времени процессора

atop - продвинутый интерактивный полноэкранный монитор производительности, написанный для Linux.

```
atop -r /var/log/atop/atop_<date> [-b hh:mm]
```

iostat/iostat

iostat - утилита, выводящая данные по использованию жесткого диска.

- `iostat -o` (активные процессы)
- `iostat -o -a` (собрать статистику за время)

iostat - утилита, предназначенная для мониторинга использования дисковых разделов.

```
iostat -d -t -p sda -x
```

- c вывести отчет по CPU
- d вывести отчет по использованию диска
- t интервал, за который усредняются значения
- x вывести расширенную статистику

Логирование



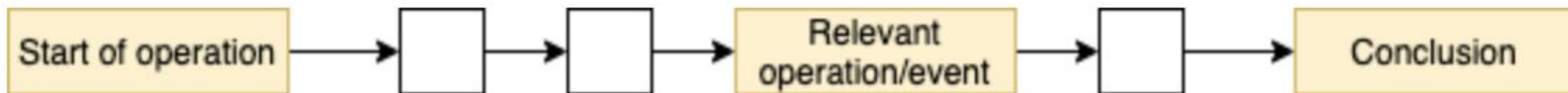
Логирование

- Отладка и диагностика
- Мониторинг и аудит
- Анализ производительности
- Исторический анализ
- Уведомления и оповещения
- Документация и отчетность

Логирование

Логи должны быть:

- Наглядными
- Контекстными
- Реактивными



Логирование: уровни

Level	Numeric value
CRITICAL	50
ERROR	40
WARNING	30
INFO	20
DEBUG	10
NOTSET	0

Логирование

- Логгеры
- Хендлеры
- Фильтры
- Форматтеры

Логирование

The basic classes defined by the module, together with their functions, are listed below.

- Loggers expose the interface that application code directly uses.
- Handlers send the log records (created by loggers) to the appropriate destination.
- Filters provide a finer grained facility for determining which log records to output.
- Formatters specify the layout of log records in the final output.

Логирование

```
import logging
logging.basicConfig(
    filename="example.log",
    level=logging.DEBUG,
)
logging.debug("This message should go to the log file")
logging.info("So should this")
logging.warning("And this, too")
```

```
$ cat example.log
DEBUG:root:This message should go to the log file
INFO:root:So should this
WARNING:root:And this, too
```

Логирование

```
import logging
```

```
logger = logging.getLogger(__name__)
```

```
def do_action(data):  
    logger.info("run do_action with %s", data)
```

Отладка



dis

```
import dis
```

```
def do_action(data):  
    logger.info("run do_action with %s", data)
```

```
dis.dis(do_action)
```

```
dis.show_code(do_action)
```

pdb

```
import pdb
```

```
def do_action(data):  
    pdb.set_trace()  
    logger.info("run do_action with %s", data)
```

```
do_action(99)
```

```
python3 -m pdb some_script.py # для всей программы целиком
```

inspect

```
import inspect
```

```
def do_action(data):  
    logger.info("run do_action with %s", data)
```

```
inspect.getmembers(do_action)
```

```
inspect.getmodule(do_action)
```

```
inspect.currentframe()
```

```
inspect.stack()
```

```
inspect.isgenerator(obj)
```

```
inspect.iscoroutine(obj)
```

```
inspect.isbuiltin(obj)
```

```
inspect.signature(do_action)
```


Домашнее задание #09

- Декоратор для профилирования
- Добавить логирование в LRUCache в отдельной копии
- По аргументу командной строки дополнительно логировать в stdout с отдельным форматом
- По аргументу применять произвольный дополнительный фильтр



Напоминание отметиться на портале Vol 2

+ ОСТАВИТЬ ОТЗЫВ ПОСЛЕ ЛЕКЦИИ

vk образование

БлогиЛюдиПрограммаВакансииРасписание

Q<

VK

Техно

Открыт приём заявок!

чт, 8 сентября

Нет занятий

пт, 9 сентября

18:00 Углубленный Py... с3
Введение в Python, основные
понятия, тестирование
Г. Кандауров

сб, 10 сентября

Нет занятий

вс, 11 сентября

Нет занятий

пн, 12 сентября

Нет занятий

Углубленный Python

↓ 0 ↑

Блог для материалов по курсу "Углубленный Python"

57 читателей, 2 топика

ПодписатьсяСоздать топик

Поиск по авторам, заголовку и тексту топика...

Найти

Добро пожаловать на курс!

Углубленный Python

ИзменитьУдалить

Всем привет и добро пожаловать на курс по углубленному изучению Python!

Прямой эфир

МоиВсе

Геннадий Кандауров час назад
Углубленный Python → Добро пожаловать
на курс! 0

Екатерина Черкасова 7 дней назад
Стажировка → Приглашаем мобильных,
фронтенд- и бэкэнд-разработчиков на
Weekend Offer! 0

Дарья Вовченко 9 дней назад
Углубленный Python → Добро пожаловать
в образовательные проекты VK
Образование! 0

Дарья Вовченко 9 дней назад
Разработка веб-сервисов на

Спасибо за
внимание

